Software Requirements Specification

for

EduTrack System

Version <1.0>

Group No.: 5

Yap Zi Xuan	**243UC247CX**
Bianca Lau Ying Xuan	**242UC2426R**
Teh Shin Rou	**243UC2467K**
Nyiam Zi Qin	**243UC2466T**

Date: 14/12/2025

Contents

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	All members	Initial creation of the Software Requirements Specification (SRS) documents.	14/12/2025

1 Project Introduction

The Smart Classroom Attendance & Analytics System is a comprehensive digital platform designed to automate and modernize the process of tracking student attendance in an academic environment. The system replaces outdated, manual methods like paper rollcalls and spreadsheet entries, which are time-consuming, prone to errors, and inefficient.

The background for this project stems from the common challenges faced by educational institutions: the loss of valuable teaching time to administrative tasks, the inaccuracy of self-reported or proxy attendance, and the inability to leverage attendance data for proactive student support.
The key problems this system aims to solve are:

1. Inefficiency in the attendance recording process.
2. Data inaccuracy and vulnerability to fraud.
3. The lack of real-time, actionable insights into student engagement patterns.

By utilizing technologies such as QR codes and geolocation for secure check-ins, and providing integrated analytics dashboards, this system seeks to streamline administrative workload, ensure data integrity, and empower educators with the tools to improve student outcomes.

1.1 Team Members

Name	Actor/Processes
Yap Zi Xuan	Programme Coordinator
Bianca Lau Ying Xuan	Admin
Teh Shin Rou	Lecturer
Nyiam Zi Qin	Student

1.2 Problem statement

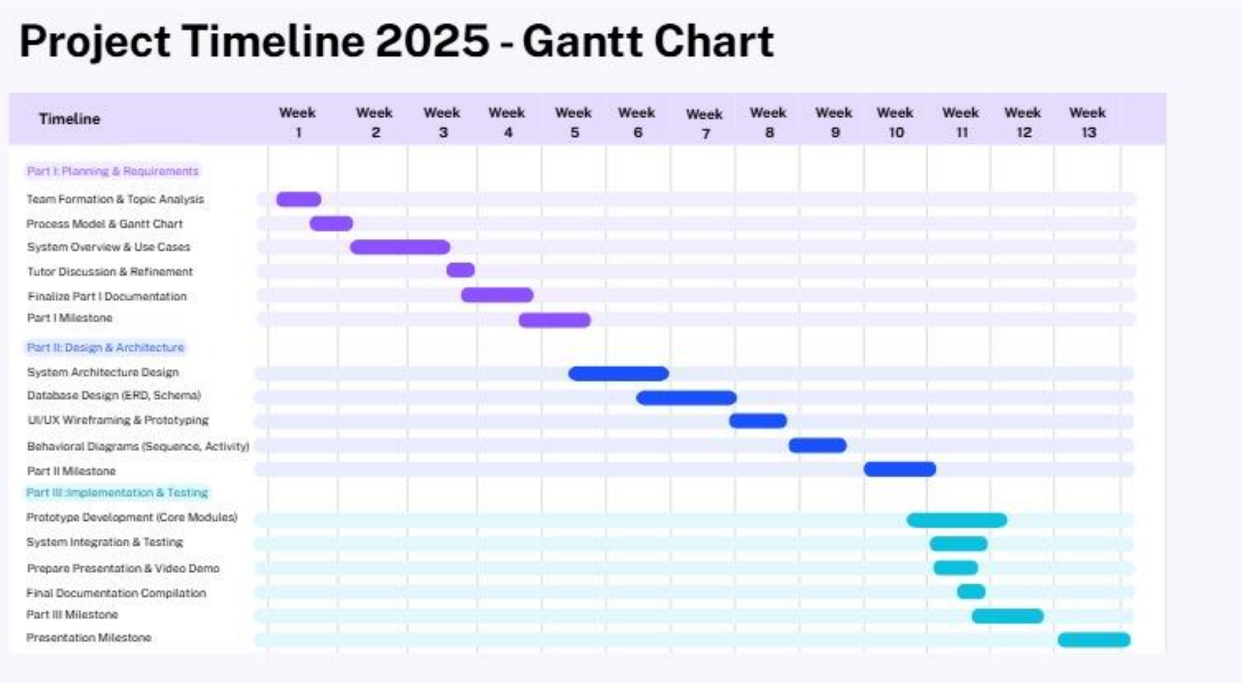
Traditional, manual methods of recording classroom attendance such as paper-based roll calls or static spreadsheets are inefficient, prone to human error, and difficult to scale. These processes consume valuable instructional time, are vulnerable to proxy attendance, and generate data that is siloed and not readily available for analysis.

Consequently, lecturers and academic administrators lack real-time, actionable insights into student attendance patterns. This makes it challenging to proactively identify and support at-risk students with declining attendance, to ensure compliance with institutional attendance policies, and to generate accurate reports for academic audits.

There is a critical need for an automated, reliable, and integrated system that streamlines the attendance-taking process, eliminates manual inefficiencies, and transforms raw attendance data into actionable intelligence to improve student engagement and academic outcomes.

1.3 Project Schedule

The project will follow an Iterative Development Model, aligned with the three main submission parts. The Gantt chart below outlines the key activities, milestones, and timeline for the project from inception to final delivery.



2 System Overview

2.1 Description

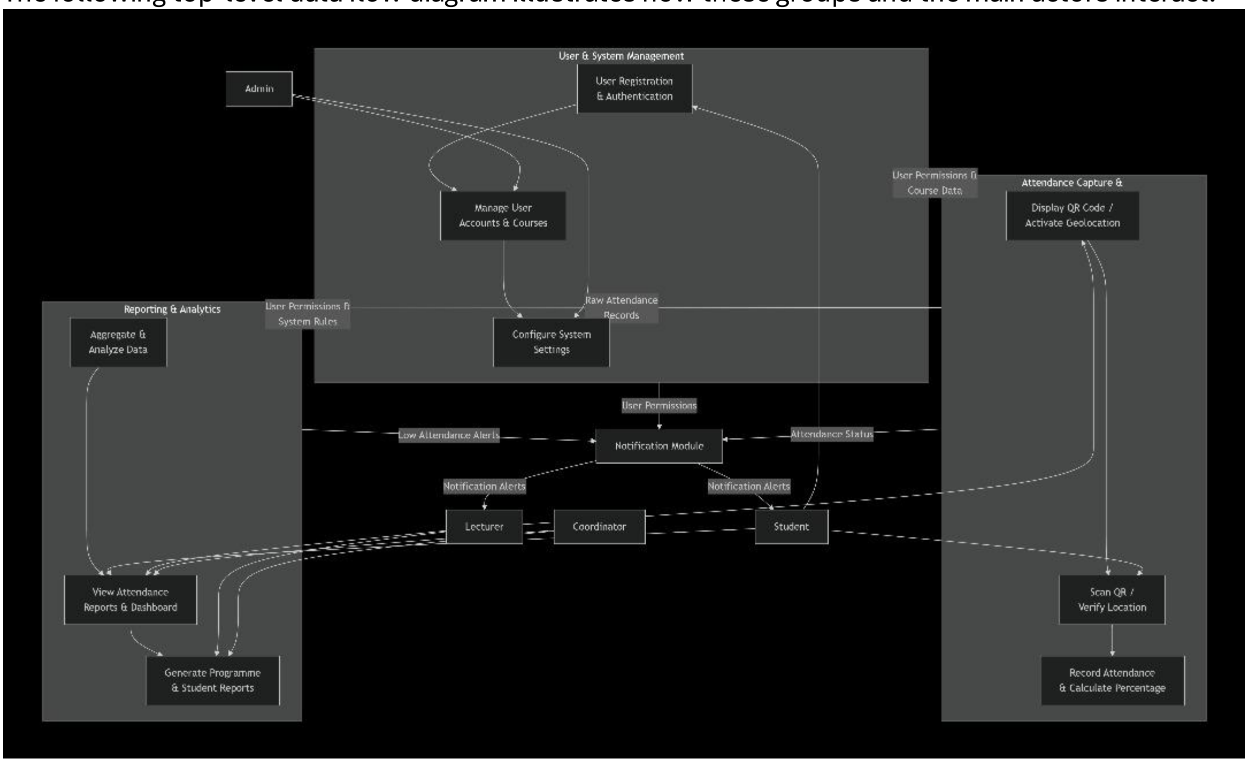
The Smart Classroom Attendance & Analytics System is designed to automate and streamline the entire process of recording, managing, and analyzing student class attendance. The system will replace error-prone, manual methods with a digital solution centered around two primary technologies: QR code scanning and geolocation verification. This ensures a fast, secure, and accurate check-in process.
The core functionality is divided into three main groups:

1. Attendance Capture & Management: This group handles the entire attendance lifecycle. It allows lecturers to create and manage class sessions, generate time-sensitive QR codes, and define geolocation boundaries. Students use this module to mark their attendance securely. The system then automatically processes this data to record presence, lateness, or absence.

2. Data Analytics & Reporting: This group transforms raw attendance data into actionable insights. It provides interactive dashboards for lecturers to monitor class and individual student attendance trends. It also enables administrators and programme coordinators to generate comprehensive reports for auditing and strategic decision-making.

3. System & User Administration: This group forms the backbone of the system, managing all user accounts (Students, Lecturers, Administrators, Programme Coordinators), configuring system-wide settings (e.g., lateness thresholds, notification rules), and maintaining data integrity.

The following top-level data flow diagram illustrates how these groups and the main actors interact:



2.2 Actors

Actor	Use Cases
Programme Coordinator	• View programme through wide analytics dashboard • Generate attendance reports for all courses in their programme • Manage notification rules for programme alerts
Admin	• Manage users account and permissions • Manage course information • Configure system settings
Lecturer	• Take attendance (display QR code/geolocation) • View students' attendance reports • Mark student's status manually (Late/Excuse)
Student	• User Registration and Authentication • Scan QR code / use geolocation to record attendance • View Personal Attendance Percentage report • Class and Timetable Management • Receive Notifications

2.3 Assumptions and Dependencies

ASSUMPTIONS:

1. All users have a stable internet connection.

- If users experience unstable connections, they may fail to submit data and take attendance.

2. The system will support a maximum of 50 concurrent users.

- If the user load increases, the system may experience slow response time or crashes.

3. The system depends on a third-party API that is always available, such as a location service.

- If the API is slow or becomes unavailable, the services could fail, and the system might require fallback APIs.

4. The system can only run on Windows Server.

- Deployment delays will occur, or the system will require a major reconfiguration.

5. The client provides adequate hardware for the system to run.

- The system may degrade or may not function properly.

6. The notification delivery depends on valid contact information.

- The system may make a mistake when marking the student's status.

7. All users can navigate the interface and follow the instructions for taking attendance.

- The system would have to be redesigned to provide additional guidance or tutorials for new users.

DEPENDENCIES:

1. User Registration and Login

- The system needs a secure place to store users' accounts, so everyone has a unique login.
- It depends on whether students, lecturers, and staff have the correct IDs.
- Passwords must be kept secret, stored in a safe, coded form, and protected with strict rules to prevent unauthorized access.

2. Attendance via QR code or Geolocation

- Attendance tracking is only available if students are registered for the class.
- Students must have devices that can scan QR codes or share location.
- The system needs to update attendance in real-time to reflect who is present.

3. Absence and Late Marking Automation
- The system can mark students absent or late only if it has the attendance record.
- Must specify what counts as "late" or "absent".

4. Attendance Analytics Dashboard
- The dashboard can only show data if attendance has been recorded.
- Only lecturers and admins can see all attendance data.
- Students can view their own attendance percentage only if they are not barred, for example, if they have paid their student fees.

5. Class and Timetable Management
- The system can only manage classes if students and lecturers are already registered
- It needs a complete and clear schedule of courses to organize classes.
- Programme coordinator or admins need to submit the timetable for the system to work

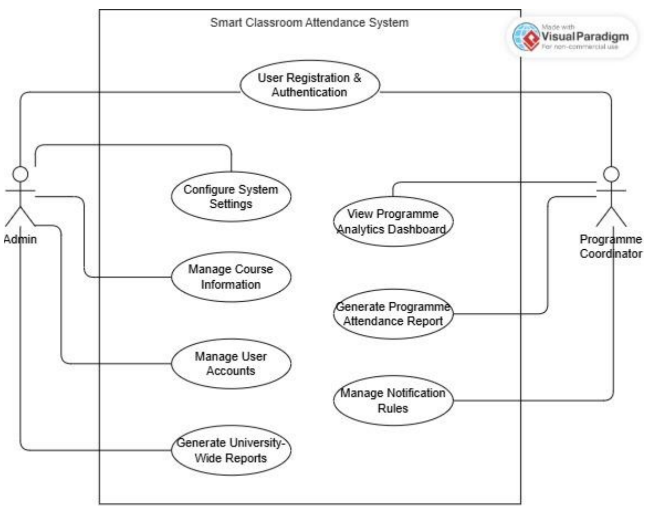
6. Export Reports
- The system can generate reports only if it has complete attendance data.
- It needs tools to create files like PDF.
- Only lecturers and admins can export reports.

7. Notification Module
- Notifications depend on attendance data
- The system needs valid contact information, such as email to send alerts.

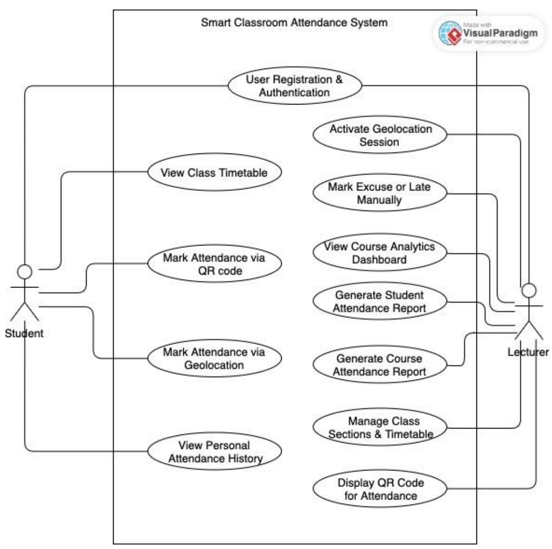
8. System Settings and User Management (Admin)
- Admins can manage users and settings only if they are properly logged in.
- The system needs to know who the admin is to allow access to sensitive options.

2.4 Use Case Diagram

Programme Coordinator & Admin



Lecturer & Student



3 Functional Requirements

3.1 Programme Coordinator

3.1.1 View Programme-Wide Analytic Dashboard

<table>
<tr><td colspan="2">Use Case Name</td><td>View Programme-Wide Analytic Dashboard</td></tr>
<tr><td colspan="2">Actors</td><td>Programme Coordinator</td></tr>
<tr><td colspan="2">Preconditions</td><td>1. User is authenticated as a Programme Coordinator.
2. Programme Coordinator has assigned courses in the system.
3. Attendance data exists for at least one course in the programme.
4. The selected period is valid.</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>Step 1: Programme Coordinator logs into the system.
Step 2: System displays the main dashboard with navigation menu.
Step 3: Programme Coordinator selects "Programme Analytics" from the dashboard.
Step 4: System retrieves attendance data for all courses under the coordinator's programme.
Step 5: System displays aggregated analytics including:
- Overall attendance percentage
- Trend graphs (weekly/monthly)
- Course-wise comparison
- At-risk student identification
Step 6: Programme Coordinator can filter by date range, course, or student group.
Step 7: System updates the dashboard in real-time based on filters.</td></tr>
<tr><td>Postconditions</td><td>1. Programme Coordinator has accessed and viewed the programme-wide analytics dashboard.
2. Dashboard data reflects the most current attendance information.
3. Coordinator can make data-driven decisions for academic planning and student support.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>1. No attendance data available
If no attendance data exists for the selected period, system displays message: “No attendance data available for the selected timeframe.”
2. Network connection lost
System shows error message and provides “Retry” option.
3. Unauthorized access attempt
System logs the attempt and redirects to login page.</td></tr>
<tr><td colspan="2">Non-functional requirements</td><td>1. Performance: Dashboard should load within 3 seconds.
2. Reliability: System should be available 99% during working hours.
3. Usability: Dashboard should be intuitive with clear visualizations.
4. Security: Only authorized Programme Coordinators can access programme-specific data.</td></tr>
</table>

3.1.2 Generate Programme Attendance Reports

Use Case Name		Generate Programme Attendance Reports
Actors		Programme Coordinator
Preconditions		1. User is authenticated as Programme Coordinator. 2. Coordinator has viewing access to programme's courses. 3. Attendance data exists for the reporting period.
Normal Flow	Description	Step 1: Programme Coordinator navigates to "Reports" section. Step 2: System displays report generation options. Step 3: Coordinator selects report type like "Programme Attendance Summary". Step 4: Coordinator sets parameters like date range, course selection, format. Step 5: System validates parameters and retrieves data. Step 6: System generates report in selected format like PDF or Excel. Step 7: Coordinator can preview, download, or share the report.
	Postconditions	1. Report is generated and saved in system. 2. Report is available for download and institutional use. 3. Student compliance and engagement metrics are documented.
Alternative flows and exceptions		1. No data for selected period System prompts to adjust date range. 2. File generation error System logs error and notifies admin. 3. Storage limit reached System alerts coordinator to delete old reports.
Non-functional requirements		1. Performance: Report generation within 30 seconds for 1000+ records. 2. Security: Reports contain sensitive data - must be encrypted in storage/transit. 3. Compatibility: Reports exportable to PDF and Excel formats.

3.1.3 Manage Notification Rules for Programmes Alerts

Use Case Name		Manage Notification Rules for Programme Alerts
Actors		Programme Coordinator
Preconditions		1. User is authenticated as Programme Coordinator. 2. Coordinator has administrative privileges for notification settings. 3. Notification module is active in the system.
Normal Flow	Description	Step 1: Programme Coordinator navigates to "Settings" → "Notification Rules". Step 2: System displays current notification rules with status. Step 3: Coordinator selects action: Create, Edit, Activate/Deactivate, or Delete rule. Step 4: For new/edit, system shows configuration form. Step 5: Coordinator sets: Rule Name, Trigger Condition, Affected Stakeholders, Notification Method, Message Template. Step 6: Coordinator saves the rule. Step 7: System validates and saves rule. Step 8: Coordinator can test rule with "Send Test Notification".
	Postconditions	1. Notification rule is saved in system. 2. Rule becomes active immediately (if enabled). 3. System will trigger notifications based on conditions. 4. Rule changes are logged for audit.
Alternative flows and exceptions		1. Rule conflict detected System warns: "Rule conflicts with existing rule [Name]." 2. Invalid condition syntax System shows: "Invalid condition format. Check syntax." 3. No stakeholders selected System prompts: "Select at least one stakeholder group." 4. Test notification fails System shows: "Test failed. Check configuration." 5. Too many active rules System advises: "Maximum active rules reached. Deactivate some first."
Non-functional requirements		1. Reliability: 99.5% successful notification delivery rate. 2. Performance: Process attendance data and trigger notifications within 5 minutes. 3. Usability: Provide pre-built templates and condition builder. 4. Security: Ensure data privacy and role-based access.

3.2 Admin

3.2.1 Manage Users Account and Permission

Use Case Name		Manage Users Account and Permission
Actors		Admin
Preconditions		1. Admin is authenticated and logged into the system. 2. System database for storing credentials and roles is accessible.
Normal Flow	Description	Step 1: Admin navigates to the User Management page. Step 2: Admin selects an action: Create, Edit, Delete/Deactivate, Assign Role, or Reset Password. Step 3: Admin enters or updates user information (name, email, role, status). Step 4: System validates the provided information. Step 5: System updates the user account in the database. Step 6: System displays a confirmation message to Admin.
	Postconditions	1. User account is updated successfully 2. New role permissions take effect immediately.
Alternative flows and exceptions		1. Duplicate Email Detected System alerts Admin that the email already exists; user creation is rejected. 2. Invalid User Input System request Admin to fix errors. 3. Database Error System logs error and displays alert. 4. Unauthorized Action System denies request.
Non-functional requirements		1. Usability: Admin interface must be intuitive with clear role labels. 2. Performance: User search must return results within 3 seconds. 3. Security: Passwords stored securely with hashing; access restricted by role. 4. Reliability: Database operations must have rollback protection. 5. Auditability: All user modifications should be logged.

3.2.2 Manage Course Information

Use Case Name		Manage Course Information
Actors		Admin
Preconditions		1. Course database exists. 2. Timetable system is integrated with the platform.
Normal Flow	Description	Step 1: Admin opens the Course Management module. Step 2: Admin selects an action: Create, Edit, or Delete a course. Step 3: Admin enters or updates course information (course code, title, credits, lecturer, timetable, classroom). Step 4: System validates the course data. Step 5: System updates the course database. Step 6: Updated course information becomes visible to all relevant users.
	Postconditions	1. Course information is updated successfully in the database. 2. Lecturer assignment, classroom details, timetable settings and attendance policies take effect immediately.
Alternative flows and exceptions		1. Course Code Already Exists System rejects the creation and requests a different code. 2. Lecturer Not Found Admin must assign an existing lecturer. 3. Invalid Timetable Entry System warns about overlapping or invalid slots. 4. Database Communication Error System logs the error and shows an alert.
Non-functional requirements		1. Performance: Course updates must reflect on all user dashboards within 5 seconds. 2. Data Integrity: Prevent overlapping room allocations. 3. Security: Only Admin role can modify course records. 4. Availability: System must support peak usage during semester registration.

3.2.3 Configure System Settings

Use Case Name		Configure System Settings
Actors		Admin
Preconditions		1. Admin privileges are verified.
Normal Flow	Description	Step 1: Admin opens the System Settings panel. Step 2: Admin selects a setting category (lateness threshold, attendance deadlines, notifications, QR code expiry, report format). Step 3: Admin adjusts the configuration values. Step 4: System validates the new configuration. Step 5: System saves and applies the new settings system-wide. Step 6: System displays a confirmation message.
	Postconditions	1. Updated rules are saved and applied system wide.
Alternative flows and exceptions		1. Invalid Settings Value System alerts Admin (e.g., QR code expiry too short). 2. Permission Denied If Admin lacks high-level permission, changes are blocked. 3. System Configuration Save Error System logs the error and retains previous settings.
Non-functional requirements		1. Reliability: System should ensure settings rollback if update fails. 2. Security: Only Admin with high-level privilege can modify global settings. 3. Audit: All configuration changes recorded with timestamp. 4. Performance: Settings must propagate within 2 seconds system wide.

3.3 Lecturer

3.3.1 Take Attendance (QR code/ Geolocation)

Use Case Name		Take Attendance (QR code/Geolocation)
Actors		Lecturer
Preconditions		1. Lecturer is logged into the system 2. A class session exists in the timetable 3. Students are enrolled in the course 4. Device cameras or GPS are enabled depending on attendance taken with which method
Normal Flow	Description	Step 1: Lecturer selects the class session from the system. Step 2: Lecturer chooses either QR code method or Geolocation method to start the attendance session. Step 3: The system generates a unique QR code for the session or a geolocation boundary. Step 4: Students scan the QR code or check in via geolocation. Step 5: The system verifies the student identity, timestamp, or location match if geolocation method is used. Step 6: The system marks students as present, and marks others as absent or late. Step 7: The system saves all attendance results into the database when the lecturer ends the attendance session.
	Postconditions	1. Attendance records for the session are stored. 2. The session is marked as completed. 3. Student's attendance status is recorded in the database.
Alternative flows and exceptions		1. Student late - If a student checks in after the allocated time, the system marks it as "Late". 2. Network failure -The system stores check-in attempts offline and syncs when reconnected. 3. Geolocation mismatch - If the student's device is not in the allowed boundary, the system blocks check-in. 4. QR code cannot be scanned - Lecturer manually mark the student as "Present".
Non-functional requirements		Performance: QR / Geolocation attendance must verify within 2 seconds. Reliability: Attendance data must not be lost even if network fails. Usability: QR code must be scannable Security: Prevent fake check-ins ad ensure geolocation cannot be spoofed Availability: System must be available during class hours

3.3.2 View Attendance Records

Use Case Name		View Attendance Records
Actors		Lecturer
Preconditions		1. Lecturer is authenticated 2. Attendance data exists in the system 3. Lecturer has access to the course
Normal Flow	Description	Step 1: Lecturer opens the Attendance module. Step 2: Lecturer chooses a course or specific student. Step 3: The system retrieves attendance records from database Step 4: The system displays daily attendance list, percentage of presence or absence, and charts or summaries. Step 5: Lecturer can filter the data by date, class, or student status. Step 6: Lecturer may export the data.
	Postconditions	1. Attendance data is displayed to the lecturer 2. Changes are only made if the lecturer updates the record
Alternative flows and exceptions		1. No records are found - System displays a “No attendance data available” message 2. Export failure - If export fails, the system allows the lecturer to retry
Non-functional requirements		Performance: Data must load within 3 seconds. Usability: Information should be readable and well-organized. Security: Only lecturers assigned to the course can view records Availability: Must work on both desktop and mobile

3.3.3 Mark Excuse or Late manually

<table>
<tr><td colspan="2">Use Case Name</td><td>Mark Excuse or Late Manually</td></tr>
<tr><td colspan="2">Actors</td><td>Lecturer</td></tr>
<tr><td colspan="2">Preconditions</td><td>1. Automatic attendance marking has completed with QR / Geolocation method
2. Lecturer is logged in to the system and has access to the course to edit attendance
3. Attendance record for the student exists</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>Step 1: Lecturer opens attendance for the selected class session.
Step 2: Lecturer selects a student whose status need manual update.
Step 3: Lecturer chooses a new status for the student. Example: Excused, Late, Present, Absent
Step 4: Lecturer may enter a justification for student.
Step 5: The system updates the attendance database with the new status.
Step 6: System refreshes the reports and analytics to reflect the change.</td></tr>
<tr><td>Postconditions</td><td>1. Updated attendance status is stored in the database.
2. Reports and analytics reflect the corrected data.</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>1. Record locked
- If the administration has locked past attendance, lecturer is not allowed to edit
2. System failure during update
- Changes are rolled back</td></tr>
<tr><td colspan="2">Non-functional requirements</td><td>Performance: All manual updates must be logged
Security: Only authorised lecturers/ admins may change attendance
Reliability: Changes must be saved even if network disconnects</td></tr>
</table>

3.4 Student

3.4.1 User Registration and Authentication

Use Case Name		User Registration and Authentication
Actors		Students
Preconditions		1. Student has a valid ID/email to register, or Student has an existing account to log in.
Normal Flow	Description	For successful log-in: Step 1: The student navigates to the system's log-in page. Step 2: The student enters their registered username and password. Step 3: The system verifies the credentials against the database. Step 4: The system grants access and redirects the student to their main dashboard. For successful registration: Step 1: The student navigates to the system's registration page. Step 2: The student enters required details. For example: name, id, email, desired password. Step 3: The system validates the input Step 4: The system creates the new user account and saves the credentials. Step 5: The system automatically logs in the student or prompt them to log in.
	Postconditions	1. Student is successfully logged in and can access their dashboard and functions.
Alternative flows and exceptions		1. Failed log-in - If student enters incorrect username or password 2. Failed registration - If student enters an already registered ID/email - If student fails to meet password complexity requirements.
Non-functional requirements		Performance: Log-in/Registration process must complete within 5 minutes. Security: Password must be stored encrypted and must follow a defined complexity standard. Reliability: The authentication service must have 99.9% uptime.

3.4.2 Scan QR code/ Use Geolocation to Record Attendance

<table>
<tr><td colspan="2">Use Case Name</td><td>Scan QR code/ Use Geolocation to Record Attendance</td></tr>
<tr><td colspan="2">Actors</td><td>Student and System</td></tr>
<tr><td colspan="2">Preconditions</td><td>1. Student is enrolled in the class
2. Student is physically present in the class location
3. Student has a device with a QR code scanner or GPS enabled
4. System has initiated the attendance session for the class</td></tr>
<tr><td rowspan="2">Normal Flow</td><td>Description</td><td>Step for QR code Attendance:
Step 1: Student opens the attendance function on their devices
Step 2: Student selects the option to scan QR code
Step 3: Student scans the unique QR code displayed by the instructor/system
Step 4: The system verifies the QR code’s validity such as class ID, time expiration
Step 5: The system records the attendance for the student in the class session

Step for GPS/Location Attendance:
Step 1: Student opens the attendance function on their device
Step 2: Student selects the option to mark via location
Step 3: Student’s device retrieves the current GPS coordinates
Step 4: System checks if the student’s coordinates are within predefined geofence boundary for the class location
Step 5: System records the attendance for the student in that class session.</td></tr>
<tr><td>Postconditions</td><td>1. Attendance is recorded in the system for that class session</td></tr>
<tr><td colspan="2">Alternative flows and exceptions</td><td>1. QR code invalid/expired
- If the QR code has expired or is invalid
2. Student outside geofence
- If the student’s GPS coordinates are outside the allowed class boundary
3. No Internet Connectivity
- If the student’s device cannot connect to the system
4. Duplicate Attendance
- If the student attempts to mark attendance a second time for the same session</td></tr>
<tr><td colspan="2">Non-functional requirements</td><td>Security: The QR code must be dynamic and time sensitive to prevent sharing/fraud
Perform: Location checking and attendance recording must less than 3 seconds
Usability: The attendance interface must be simple, requiring minimal taps
Accuracy: GPS location check accuracy must be within a 10-meter radius of the class location</td></tr>
</table>

3.4.3 View Personal Attendance Percentage Report

Use Case Name		View Personal Attendance Percentage Report
Actors		Student
Preconditions		1. Student is successfully logged into the system. 2. Attendance data for the student has been recorded for the relevant classes. 3. Student is not barred from viewing the report if tuition fees are paid.
Normal Flow	Description	Step 1: Student navigates to the “Attendance” or “Reports” section of the dashboard. Step 2: Student selects the option to view the “Personal Attendance Report” Step 3: The system retrieves all recorded attendance data for the student across all enrolled courses Step 4: The system calculates the attendance percentage for each course Step 5: The system generates and displays the report, typically showing course name, sessions, sessions attended, and the calculated percentage
	Postconditions	1. Student can see an accurate attendance report for their enrolled courses.
Alternative flows and exceptions		1. Barred from Viewing - If student is logged in but is barred from viewing the report. For example, pending tuition fees. 2. No Attendance Data - If student has recently enrolled, and no attendance sessions have been held yet. 3. System Error - If a connection or database error occurs during data retrieval.
Non-functional requirements		Performance: The report must load and display all relevant data within 5 seconds, regardless of the number of courses or sessions. Security: Student must only be able to view their own attendance data. Usability: The report must be presented clearly, perhaps in a table or graphical format, easily readable on both desktop and mobile devices.

3.4.4 Class and Timetable Management

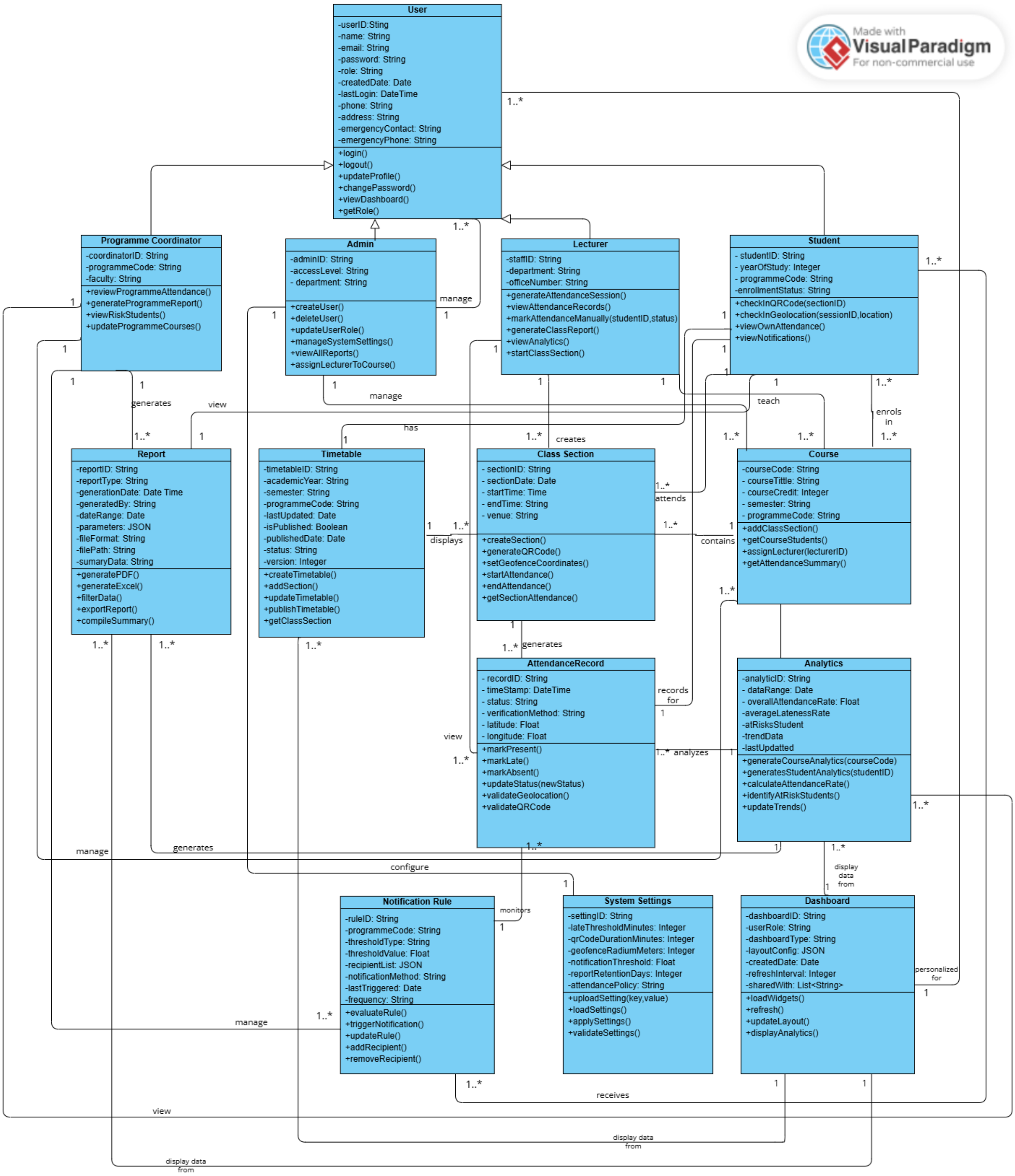
Use Case Name		Class and Timetable Management
Actors		Student
Preconditions		1. Student is successfully logged into the system. 2. Classes and timetables have been created and assigned to the student by the system administrator.
Normal Flow	Description	Step 1: Student navigates to the "Timetable" or "Schedule" section of their dashboard. Step 2: System retrieves student's enrolled courses and the corresponding schedule. Step 3: System compiles and display the timetable. For example: daily or weekly schedule. Step 4: Student can optionally filter the schedule. Fo example: day, week or course.
	Postconditions	1. Student can see an accurate daily or weekly class schedule.
Alternative flows and exceptions		1. No Assigned Schedule - If student has enrolled, but the timetable has not yet finalized or assigned. 2. Filtering/Viewing Error - If student attempts to apply a filter that returns no results. For example: viewing next semester's schedule before it's released. 3. Data Retrieval Failure - If connection or database error occurs during data retrieval.
Non-functional requirements		Performance: The schedule must load and display within 3 seconds. Maintainability: Schedule data structure must allow for easy updates when class times or locations are changed by the administrator. Usability: Timetable display must be clear, easy to read and responsive on mobile devices.

3.4.5 Receive Notifications

Use Case Name		Receive Notification
Actors		Student and System
Preconditions		1. Student is registered in the system 2. Student has valid contact information such as email address and has in app notification preference enabled. 3. Notification rules have been configured by the administrator.
Normal Flow	Description	Step 1: A predefined system event or trigger occurs. For example: the system detects a student's attendance for a course has dropped below 80%. Step 2: System generates the appropriate notification content. Step 3: System pushes the notification via the determined channel. Step 4: Student receives and views the notification.
	Postconditions	1. Student receives notifications relevant to their courses and attendance status.
Alternative flows and exceptions		1. Invalid Contact Information - If student's registered email address is invalid or bounced back. 2. Notification Preferences Disabled - Student has disabled a specific notification channel. 3. System Overload - If high volume of notifications is triggered simultaneously causing a delay in delivery.
Non-functional requirements		Reliability: Notifications must be delivered accurately and promptly. Security: Notification content must not expose sensitive personal information unnecessarily. Maintainability: The system must allow administrators to easily modify or add new notification trigger conditions and message templates. Usability: The in-app notification center should clearly display the time, source, and severity of each alert.

4 System Models

4.1 Class Diagrams



4.2 Classes

Class	Description
User	Base entity for all system actors, provide core authentication and account details.
Programme Coordinator	Manages and oversees a specific programme and its associated course.
Admin	Responsible for configuring system setting and managing user accounts.
Lecturer	Responsible for teaching courses and managing class sessions.
Student	Represents a learner enrolled in a programme with study progress information.
Course	Defines a subject offered by the institution.
Class Session	Represent a scheduled class meeting for a course.
Attendance Record	Logs a student's attendance status for a specific class session.
Timetable	Show student's weekly schedule containing enrolled course sessions.
Analytics	Aggregates attendance data to generate insights.
Report	Generate documents summarizing attendance or analytics based on given parameters.
System Settings	Stores configurable system parameters.
Dashboard	Display personalized widgets and data summaries for specific user.
Notification Rule	Defines automated alert rules based on thresholds for attendance or performance metrics.

Class Relationships and Cardinality

From Class	Relationship	To Class	Cardinality
User	generalizes	Programme Coordinator, Admin, Lecturer, Student	1 → 1..*
Programme Coordinator	manages	Courses	1 → 1..*
Programme Coordinator	views	Analytics	1 → 1..*
Programme Coordinator	generates	Report	1 → 1..*
Programme Coordinator	manages	Notification Rule	1 → 1..*
Admin	manages	User	1 → 1..*
Admin	configures	System Settings	1 → 1
Admin	manages	Course	1 → 1..*
Lecturer	teaches	Course	1 → 1..*
Lecturer	creates	Class Section	1 → 1..*
Lecturer	views	Attendance Record	1 → 1..*
Student	attends	Class Section	1 → 1..*
Student	has	Timetable	1 → 1
Student	views	Report	1 → 1
Student	receives	Notification	1..* → 1..*
Student	enrols in	Course	1..* → 1..*
Course	contains	Class Section	1 → 1..*
Class Section	generates	Attendance Record	1 → 1..*
Analytics	analyzes	Attendance Record	1 → 1..*
Analytics	generates	Report	1 → 1..*
Attendance Record	records for	Student	1 → 1
Timetable	displays	Class Section	1 → 1..*
Notification Rule	monitors	Attendance Record	1 → 1..*
Dashboard	Personalized for	User	1 → 1..*
Dashboard	Displays data from	Analytics, Report, Timetable	1 → 1..*

5 References

Visual Paradigm International Ltd. (n.d.). Creating a class diagram. Visual Paradigm. Retrieved December 10, 2025, from https://www.visual-paradigm.com/support/documents/vpuserguide/94/2576/7190_creatingclass.html

Visual Paradigm International Ltd. (n.d.). Creating a use case diagram. Visual Paradigm. Retrieved December 10, 2025, from https://www.visual-paradigm.com/support/documents/vpuserguide/94/2575/6362_creatingusec.html

Visual Paradigm International Ltd. (n.d.). Drawing activity and swimlane diagrams. Visual Paradigm. Retrieved December 10, 2025, from https://www.visual-paradigm.com/support/documents/vpuserguide/94/2580/6713_drawingactiv.html

Visual Paradigm International Ltd. (n.d.). Writing effective use cases. Visual Paradigm. Retrieved December 10, 2025, from https://www.visual-paradigm.com/tutorials/writingeffectiversecase.jsp

Visual Paradigm International Ltd. (2024). Visual Paradigm Online [Software]. https://online.visual-paradigm.com